A short note on NULLs

As promised in the last lesson, we are going to quickly talk about **NULL** values in an SQL database. It's always good to reduce the possibility of **NULL** values in databases because they require special attention when constructing queries, constraints (certain functions behave differently with null values) and when processing the results.

An alternative to **NULL** values in your database is to have **data-type appropriate default values**, like 0 for numerical data, empty strings for text data, etc. But if your database needs to store incomplete data, then **NULL** values can be appropriate if the default values will skew later analysis (for example, when taking averages of numerical data).

Sometimes, it's also not possible to avoid **NULL** values, as we saw in the last lesson when outer-joining two tables with asymmetric data. In these cases, you can test a column for **NULL** values in a **WHERE** clause by using either the **IS NULL** or **IS NOT NULL** constraint.

Select query with constraints on NULL values

```
SELECT column, another_column, …
FROM mytable
WHERE column IS/IS NOT NULL
AND/OR another_condition
AND/OR …;
```

Exercise

This exercise will be a sort of review of the last few lessons. We're using the same **Employees** and **Buildings** table from the last lesson, but we've hired a few more people, who haven't yet been assigned a building.

Exercise — Tasks

1. Find the name and role of all employees who have not been assigned to a building
2. Find the names of the buildings that hold no employees